Filtering in Pandas 2

Filtering Dataframes

- Group by
  - Compute **aggregate** statistics of groups in the data
    - Compute group sums or means
  - Perform some group specific **transformations** on the data
    - Fill NaNs of groups with mean of group
  - **Filter** the data based on some group-wise operations
    - Discard data that belongs to groups with only a few data points.

Looking at mtcars Again

```
import pandas as pd

df_mtcars = pd.read_csv("../Data/mtcars.csv")
df_mtcars.head(8)
```

	car_name	mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	carb
0	Mazda RX4	21.0	6	160.0	110	3.90	2.620	16.46	0	1	4	4
1	Mazda RX4 Wag	21.0	6	160.0	110	3.90	2.875	17.02	0	1	4	4
2	Datsun 710	22.8	4	108.0	93	3.85	2.320	18.61	1	1	4	1
3	Hornet 4 Drive	21.4	6	258.0	110	3.08	3.215	19.44	1	0	3	1
4	Hornet Sportabout	18.7	8	360.0	175	3.15	3.440	17.02	0	0	3	2
5	Valiant	18.1	6	225.0	105	2.76	3.460	20.22	1	0	3	1
6	Duster 360	14.3	8	360.0	245	3.21	3.570	15.84	0	0	3	4
7	Merc 240D	24.4	4	146.7	62	3.69	3.190	20.00	1	0	4	2

Looking at mtcars Again

```
import pandas as pd

df_mtcars = pd.read_csv("../Data/mtcars.csv")
df_mtcars.head(8)
```

	car_name	mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	carb
0	Mazda RX4	21.0	6	160.0	110	3.90	2.620	16.46	0	1	4	4
1	Mazda RX4 Wag	21.0	6	160.0	110	3.90	2.875	17.02	0	1	4	4
2	Datsun 710	22.8	4	108.0	93	3.85	2.320	18.61	1	1	4	1
3	Hornet 4 Drive	21.4	6	258.0	110	3.08	3.215	19.44	1	0	3	1
4	Hornet Sportabout	18.7	8	360.0	175	3.15	3.440	17.02	0	0	3	2
5	Valiant	18.1	6	225.0	105	2.76	3.460	20.22	1	0	3	1
6	Duster 360	14.3	8	360.0	245	3.21	3.570	15.84	0	0	3	4
7	Merc 240D	24.4	4	146.7	62	3.69	3.190	20.00	1	0	4	2

What is the avg mpg for each cylinder type?

Groupby in Pandas - Aggregating

Basic Group By

```
#Group by column cyl, compute mean of each group
df_mtcars.groupby(by=["cyl"])["mpg"].mean()
```

Create groups based on
unique entries in cyl column

Basic Group By

```
#Group by column cyl, compute mean of each group
df_mtcars.groupby(by=["cyl"])["mpg"].mean()
```

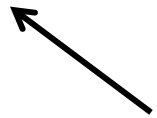
Create groups based on unique entries in cyl column

For each group compute the mean mpg

Basic Group By

```
#Group by column cyl, compute mean of each group
df_mtcars.groupby(by=["cyl"])["mpg"].mean()
```

```
cyl
4    26.663636
6    19.742857
8    15.100000
Name: mpg, dtype: float64
```



Returns a series where the indices are the groups

Basic Group By

```
#Group by column cyl, compute mean of each group
df_mtcars.groupby(by=["cyl"])["mpg"].mean()
```

```
cyl
4    26.663636
6    19.742857
8    15.100000
Name: mpg, dtype: float64
```

Returns a series where the indices are the groups

Remaining Questions:

- Can I group by more than one column?
- Can I compute more than one aggregate statistic for each group?
- For each group can I customize how I summarize each column that I select?

Selecting Multiple Columns

```
#Selecting multiple columns after grouping
df_mtcars.groupby(by=["cyl"])["mpg", "hp"].mean()
```

	mpg	hp
cyl		
4	26.663636	82.636364
6	19.742857	122.285714
8	15.100000	209.214286

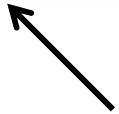
Specify the columns you want

- We get avg mpg and hp for each of the three cylinder groups.
- Since we are selecting two columns we get back a dataframe

Grouping By Multiple Columns

```
#Selecting multiple columns after grouping
df_mtcars.groupby(by=["cyl", "am"])["mpg", "hp"].mean()
```

		mpg	hp
cyl	am		
4	0	22.900000	84.666667
	1	28.075000	81.875000
6	0	19.125000	115.250000
	1	20.566667	131.666667
8	0	15.050000	194.166667
	1	15.400000	299.500000



Specify columns you want to group by

- We have a group for every combination of cyl and am.
- We get avg mpg and hp for each of the six groups.
- We get a multi-indexed dataframe (two row names).

Slicing Multi-indexed DataFrame

Options 1:

```
df_1
```

		mpg	hp
cyl	am		
4	0	22.900000	84.666667
	1	28.075000	81.875000
6	0	19.125000	115.250000
	1	20.566667	131.666667
8	0	15.050000	194.166667
	1	15.400000	299.500000

```
#Get row where cyl=4
df_1.loc[4,:]
```

	mpg	hp
am		
0	22.900	84.666667
1	28.075	81.875000

```
#Get row where cyl=6, am=1
df_1.loc[(6,1),:]
```

```
mpg      20.566667
hp      131.666667
Name: (6, 1), dtype: float64
```

Just Reset Index…

Options 2:

```
#Selecting multiple columns after grouping
df_1  = df_mtcars.groupby(by=["cyl", "am"])["mpg", "hp"].mean()
df_1
```

		mpg	hp
cyl	am		
4	0	22.900000	84.666667
	1	28.075000	81.875000
6	0	19.125000	115.250000
	1	20.566667	131.666667
8	0	15.050000	194.166667
	1	15.400000	299.500000

```
df_1.reset_index(inplace=True)
df_1
```

	cyl	am	mpg	hp
0	4	0	22.900000	84.666667
1	4	1	28.075000	81.875000
2	6	0	19.125000	115.250000
3	6	1	20.566667	131.666667
4	8	0	15.050000	194.166667
5	8	1	15.400000	299.500000

Apply Multiple Functions

```
import numpy as np

df_mtcars.groupby(by=["cyl"])["mpg"].agg([np.mean, np.std])
```

	mean	std
cyl		
4	26.663636	4.509828
6	19.742857	1.453567
8	15.100000	2.560048

List of functions I want to apply to groups

- For each group of cylinders we get the mean and stdev mpg.

Apply Multiple Functions to Multiple Columns

```
import numpy as np

df_2 = df_mtcars.groupby(by=["cyl"])["mpg", "hp"].agg([np.mean, np.std])
df_2
```

	mpg		hp	
	mean	std	mean	std
cyl				
4	26.663636	4.509828	82.636364	20.934530
6	19.742857	1.453567	122.285714	24.260491
8	15.100000	2.560048	209.214286	50.976886

- For each group of cylinders we get the mean and stdev of mpg and hp.
- Get dataframe with multi-indexed column names

Apply Multiple Functions to Multiple Columns

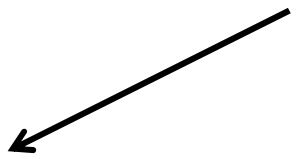
```
import numpy as np

df_2 = df_mtcars.groupby(by=["cyl"])["mpg", "hp"].agg([np.mean, np.std])
df_2
```

	mpg		hp	
	mean	std	mean	std
cyl				
4	26.663636	4.509828	82.636364	20.934530
6	19.742857	1.453567	122.285714	24.260491
8	15.100000	2.560048	209.214286	50.976886

```
#Select mpg info
df_2.loc[:, "mpg"]
```

	mean	std
cyl		
4	26.663636	4.509828
6	19.742857	1.453567
8	15.100000	2.560048



Get dataframe back

Apply Multiple Functions to Multiple Columns

Dictionary

```
df_mtcars.groupby(by=["cyl"]).agg({"hp":np.mean, "mpg":np.std})
```

	hp	mpg
cyl		
4	82.636364	4.509828
6	122.285714	1.453567
8	209.214286	2.560048

For hp column compute mean of groups and for mpg column compute standard deviation

groupby().agg(dict(column: [function(s)]))
groupby().agg({'hp': ['mean', 'median'], 'mpg': 'std'})

Apply Custom Function to Column

```
#Counts number of Toyotas a Mercedes in group
def Count_Toyota_Merc(group):
    count=0
    for index in list(group.index):
        car_name = group[index].lower()

        if "toyota" in car_name or "merc" in car_name:
            count+=1
    return count
```

Apply Custom Function to Column

```
#Counts number of Toyotas a Mercedes in group
def Count_Toyota_Merc(group):
    count=0
    for index in list(group.index):
        car_name = group[index].lower()

        if "toyota" in car_name or "merc" in car_name:
            count+=1
    return count
```

Assume group is passed as series.

```
#Use groupby with custom function
df_mtcars.groupby(by = "am").agg({"car_name": Count_Toyota_Merc})
```

	car_name
am	
0	8
1	1